卸売業BPO詳細設計

スコープ: 中小卸売業（10-300名規模、年商1-100億円）。食品/建材/機械工具/日用品卸が主要ターゲット
攻略優先度: A（コア6業界）
ゲノム系統: 建設系（OCR→抽出→照合の共通パターン）
実装状況: 未着手。受発注AIパイプラインから開発開始
制約: 基幹システム（販売管理ソフト）連携はPhase 2+。Phase 1はCSV入出力

マイクロエージェント対応表

設計書での呼称	実装関数名	ファイル
OCR/テキスト抽出	run_document_ocr	workers/micro/ocr.py
構造化抽出	run_structured_extractor	workers/micro/extractor.py
ルール照合	run_rule_matcher	workers/micro/rule_matcher.py
コスト計算	run_cost_calculator	workers/micro/calculator.py
コンプライアンスチェック	run_compliance_checker	workers/micro/compliance.py
文書生成	run_document_generator	workers/micro/generator.py
バリデーション	run_output_validator	workers/micro/validator.py
SaaS読取	run_saas_reader	workers/micro/saas_reader.py
SaaS書込	run_saas_writer	workers/micro/saas_writer.py
メッセージ生成	run_message_drafter	workers/micro/message.py
差分検出	run_diff_detector	workers/micro/diff.py
PDF生成	run_pdf_generator	workers/micro/pdf_generator.py

---

0. エグゼクティブサマリー

なぜ卸売業BPOか

```
事業所数: 約25万社
市場規模: 約400兆円/年（日本のB2B流通の中核）
DX進捗: 中小卸はFAX・電話が主流。受発注の70-80%がアナログ

中小卸売業は：
  ・受注の60-80%がFAX/電話/メール。1件10-15分の手入力
  ・月500件の受注で80-125時間の入力作業
  ・在庫問い合わせに営業が1日20-50件対応（電話番状態）
  ・得意先ごとに違う掛率・締日・支払条件の管理が地獄
  ・請求書と納品書の突合が毎月の地獄（月末3日間は経理が徹夜）
  ・仕入先からのリベート計算が複雑（年間数百万〜数千万円）
 → 建設業の「OCR→抽出→照合」と同じ構造
```

支払いロジック

```
■ 直接効果①：受注入力工数の削減
  月500件 × 10分/件 = 月83時間
  AI化で70%削減 = 月58時間削減
  → 時給1,500円 × 58時間 = 月8.7万円

■ 直接効果②：受注ミスの削減
  手入力のミス率: 2-5%（月10-25件）
  1件あたりのミスコスト: 返品・再配送で平均5,000-10,000円
  → 月5-25万円の損失削減

■ 直接効果③：請求照合の工数削減
  月200件 × 20分/件 = 月67時間
  AI化で80%削減 = 月53時間削減
  → 時給1,500円 × 53時間 = 月8万円

■ 直接効果④：在庫最適化
  在庫回転率の改善（6回→8回/年）
  平均在庫1億円の場合 → 在庫削減25% = 2,500万円の資金効率改善
  → 金利3%換算で年75万円の資金コスト削減

■ 直接効果⑤：営業の受注機会増
  在庫問い合わせ即答 → 受注率5%向上
  月商5,000万円の場合 → 月250万円の売上増（粗利率15%で37.5万円）

■ 合計効果：年500-1,200万円（月商5,000万円規模）
  → BPO月20万円（年240万円）で ROI 2.1〜5.0倍
```

6モジュール全体像

#	モジュール	Tier	月額価値	実装Phase
1	受発注AI	★キラー	¥10-20万	Phase A
2	在庫・倉庫管理	Tier1	¥5-10万	Phase A
3	請求・売掛管理	Tier1	¥5-10万	Phase B
4	仕入・買掛管理	Tier1	¥3-5万	Phase B
5	物流・配送管理	Tier2	¥3-5万	Phase C
6	営業支援（卸売特化）	Tier2	¥3-5万	Phase C

---

1. 受発注AI（★キラーフィーチャー）

1.1 問題の本質

```
受発注の現状:
  1. 得意先がFAX/メール/電話/LINEで注文を送ってくる
  2. 事務員がFAXを見て、販売管理ソフトに手入力
  3. 商品名の表記揺れ（略称、型番違い、旧品番）に悩まされる
  4. 在庫を確認して、不足分は仕入先に発注
  5. 受注確認書をFAX/メールで返信
  6. ピッキングリスト → 出荷 → 納品書・送り状

  → 事務員2-3名がフル稼働で月500-1,000件の受注処理
  → FAXの読み間違い → 誤出荷 → 返品・再配送コスト
  → 電話の聞き間違い → 数量ミス → 信頼失墜
  → 注文の集中する月末・月初は残業地獄
```

1.2 技術設計 — 3段階アプローチ

```
■ Level 1: OCR + LLM で注文内容を構造化抽出（手入力70%削減）
  入力: FAX画像(TIFF/PDF)、メール本文、LINEスクショ
  処理:
    ① Google Document AI でテキスト抽出（日本語OCR精度: 手書き80-90%、活字95%+）
    ② LLM で構造化抽出（商品名・数量・単位・納期・発注元）
    ③ 確信度スコアリング（0.0-1.0）
    ④ 確信度 0.85未満 → HITL（人間確認キューに投入）
  出力: 構造化注文データ

  精度目標:
    - 活字FAX: 95%以上（自動処理可能率）
    - 手書きFAX: 80%以上（残りはHITL）
    - メール: 98%以上（テキストベースなのでOCR不要）

■ Level 2: 商品マスタとのファジーマッチ（表記揺れ対応）
  処理:
    ① 完全一致（商品コード/JANコード）→ confidence=1.0
    ② 前方一致/部分一致（商品名の一部）→ confidence=0.8-0.9
    ③ 類似度検索（Levenshtein距離 + Embedding類似度）→ confidence=0.6-0.8
    ④ 得意先別の注文履歴パターン（「いつもの」対応）→ confidence=0.7-0.9
    ⑤ 候補なし → HITL

  表記揺れ辞書（自動学習）:
    - 「ワイドハイター」→「花王 ワイドハイターEX 詰替 480ml (4901301295095)」
    - 「WH詰替」→ 同上
    - 「いつもの洗剤」→ 得意先Aの場合は上記（過去注文パターンから推定）

■ Level 3: 取引先別注文パターン学習 → 予測入力
  処理:
    ① 定期注文パターンの検出（毎週月曜に同じ商品を発注する得意先）
    ② 季節変動の学習（夏に飲料が増える等）
    ③ 自動受注提案（「田中商店は月曜なのに未注文です。確認しますか？」）
```

1.3 パイプラインステップ

Step	マイクロエージェント	入力	処理	出力
1	run_rule_matcher	raw_document (FAX/メール/LINE)	注文ドキュメント受信・分類（種別判定+発注元特定）	classified_document
2	run_document_ocr	classified_document (FAX/画像)	OCRテキスト抽出（Google Document AI。メールはスキップ）	extracted_text
3	run_structured_extractor	extracted_text	注文内容の構造化抽出（LLM。商品名/数量/単位/納期+確信度スコア）	structured_order
4	run_rule_matcher	structured_order + product_master	商品マスタ照合（完全一致→ファジー→Embedding→履歴パターン）	matched_products
5	run_cost_calculator	matched_products + inventory_data	在庫確認・引当（有効在庫算出→不足分発注提案→納期回答生成）	inventory_result
6	run_document_generator	inventory_result	受注確認書自動生成・返信（元の注文方法に合わせた返信チャネル選定）	confirmation_doc
7	run_output_validator	confirmation_doc	バリデーション（必須フィールド完全性/数量異常値/与信限度額チェック）	OrderProcessingResult

1.4 パイプライン詳細設計（実装）

```
# workers/bpo/wholesale/pipelines/order_processing_pipeline.py

class OrderProcessingPipeline:
    """
    卸売業 受発注AIパイプライン

    Step 1: document_receiver       注文ドキュメント受信・分類
      - マイクロエージェント: micro/document_classifier
      - 入力: FAX画像(TIFF/PDF)、メール本文、LINE画像
      - 処理:
        (a) ドキュメント種別判定（注文書/見積依頼/問い合わせ/その他）
        (b) 発注元の特定（FAX番号→得意先マスタ照合、メールアドレス→同）
        (c) 注文書以外はルーティング（見積依頼→見積パイプライン等）
      - 出力: 分類済みドキュメント + 発注元情報

    Step 2: ocr_extractor           OCRテキスト抽出
      - マイクロエージェント: micro/ocr_reader（Google Document AI）
      - 処理:
        (a) FAX/画像 → テキスト抽出
        (b) テーブル構造の検出（表形式の注文書に対応）
        (c) 手書き文字の認識（活字/手書きの混在対応）
        (d) メール本文はこのステップをスキップ
      - 出力: 抽出テキスト + レイアウト情報

    Step 3: order_structurer        注文内容の構造化抽出
      - マイクロエージェント: micro/extractor（LLM活用）
      - LLMプロンプト:
          「以下の注文書テキストから、注文情報を構造化して抽出してください。
           抽出項目: 発注元名、発注日、納品希望日、商品名、商品コード（あれば）、
           数量、単位、単価（あれば）、備考
           確信度が低い項目には confidence: low を付与してください。」
      - 処理:
        (a) LLMによる構造化抽出
        (b) 各フィールドの確信度スコアリング
        (c) 数量の単位正規化（ケース/箱/本/個/kg/L）
            - 「3C」→ 3ケース
            - 「ダース」→ 12個
            - 「半ダース」→ 6個
      - 出力: 構造化注文データ（確信度付き）

    Step 4: product_matcher         商品マスタ照合（ファジーマッチ）
      - マイクロエージェント: micro/product_matcher
      - 処理:
        (a) 商品コード/JAN完全一致 → confidence=1.0
        (b) 商品名の正規化（全角→半角、カタカナ→ひらがな等）
        (c) Levenshtein距離 ≤ 3 → confidence=0.8
        (d) Embedding類似度 ≥ 0.85 → confidence=0.7
        (e) 得意先別注文履歴パターンマッチ → confidence=0.75
        (f) 複数候補 → 候補リストをHITLに提示
```

```
      - マッチング式:
        total_confidence = w1×exact_match + w2×fuzzy_score + w3×embedding_sim +
w4×history_match
        w1=0.4, w2=0.25, w3=0.2, w4=0.15
      - 出力: マッチ済み商品リスト + 未マッチ商品リスト

    Step 5: inventory_checker       在庫確認・引当
      - マイクロエージェント: micro/inventory_checker
      - 処理:
        (a) 商品別の在庫確認
            有効在庫 = 現在庫 - 引当済み数量 + 入荷予定数量
        (b) 在庫充足判定
            有効在庫 ≥ 注文数量 → 引当OK
            有効在庫 < 注文数量 → 不足分を計算
        (c) 不足分の自動発注提案
            不足数量 = 注文数量 - 有効在庫
            → 仕入先マスタから最適仕入先を選定（価格/リードタイム/最低ロット）
        (d) 納期回答の生成
            在庫あり → 翌営業日出荷
            不足分あり → 仕入先リードタイム + 1日
      - 出力: 引当結果 + 発注提案 + 納期回答

    Step 6: confirmation_generator  受注確認書自動生成・返信
      - マイクロエージェント: micro/document_generator
      - 処理:
        (a) 受注確認書の生成（注文内容の確認 + 納期回答）
        (b) 返信方法の選定（元の注文方法に合わせる）
            - FAX受注 → FAX返信
            - メール受注 → メール返信
            - LINE受注 → LINE返信
        (c) 確信度 < 0.85 の項目がある場合 → 確認事項を明記
      - 出力: 受注確認書（PDF/メール）

    Step 7: output_validator        バリデーション・最終出力
      - マイクロエージェント: micro/validator
      - 処理:
        (a) 必須フィールドの完全性チェック
        (b) 数量の異常値チェック（過去平均の5倍超 → 確認アラート）
        (c) 与信限度額チェック（受注累計が与信限度額の90%超 → アラート）
        (d) HITL対象の集約（要確認項目の一覧）
      - 出力: OrderProcessingResult
    """
```

1.4 データモデル

```
// 入力: 注文ドキュメント
{
  "document_id": "DOC-20260328-001",
  "source_type": "fax",
  "source_identifier": "03-1234-5678",
  "received_at": "2026-03-28T09:15:00",
  "document_url": "gs://bucket/fax/20260328_091500.tiff",
  "customer_id": "CUS-001",
  "customer_name": "田中商店"
}

// Step 3 出力: 構造化注文データ
{
  "order_id": "ORD-20260328-001",
  "customer_id": "CUS-001",
  "customer_name": "田中商店",
  "order_date": "2026-03-28",
  "desired_delivery_date": "2026-03-30",
  "items": [
    {
      "line_no": 1,
      "raw_text": "ワイドハイター詰替 3C",
      "product_code": null,
      "product_name_extracted": "ワイドハイター詰替",
      "quantity": 3,
      "unit": "ケース",
      "unit_price": null,
      "confidence": {
        "product_name": 0.92,
        "quantity": 0.98,
        "unit": 0.85
      }
    },
    {
      "line_no": 2,
      "raw_text": "花王キュキュット 5",
      "product_code": null,
      "product_name_extracted": "花王キュキュット",
      "quantity": 5,
      "unit": "個",
      "confidence": {
        "product_name": 0.88,
        "quantity": 0.95,
        "unit": 0.60
      }
    }
  ],
  "notes": "午前中着希望",
  "overall_confidence": 0.86
```

```
}

// Step 4 出力: 商品マッチ結果
{
  "matched_items": [
    {
      "line_no": 1,
      "matched_product": {
        "product_id": "PRD-4901301295095",
        "product_code": "WH-EX-480R",
        "product_name": "花王 ワイドハイターEX パワー 詰替 480ml",
        "jan_code": "4901301295095",
        "unit_price": 298,
        "case_quantity": 24
      },
      "match_type": "fuzzy_name",
      "match_confidence": 0.88,
      "quantity_in_pieces": 72,
      "amount_yen": 21456
    }
  ],
  "unmatched_items": [],
  "hitl_required": [
    {
      "line_no": 2,
      "reason": "単位不明（個 or ケース)。候補: キュキュット泡スプレー/キュキュット本体/キュキュット詰
替",
      "candidates": [
        {"product_id": "PRD-001", "product_name": "キュキュット クリア泡スプレー 本体 300ml",
"confidence": 0.72},
        {"product_id": "PRD-002", "product_name": "キュキュット 食器用洗剤 本体 240ml",
"confidence": 0.68},
        {"product_id": "PRD-003", "product_name": "キュキュット 食器用洗剤 詰替 385ml",
"confidence": 0.65}
      ]
    }
  ]
}

// 最終出力: 受注確認書データ
{
  "confirmation_id": "CNF-20260328-001",
  "order_id": "ORD-20260328-001",
  "customer_name": "田中商店",
  "items": [
    {
      "product_name": "花王 ワイドハイターEX パワー 詰替 480ml",
      "quantity": "3ケース（72個）",
      "unit_price": 298,
      "amount": 21456,
      "availability": "在庫あり",
      "estimated_delivery": "2026-03-30"
```

```
    }
  ],
  "subtotal": 21456,
  "tax": 2146,
  "total": 23602,
  "delivery_method": "自社便",
  "payment_terms": "月末締め翌月末払い",
  "notes": "2行目のキュキュットについて確認のご連絡をいたします"
}
```

1.5 制約・前提条件

```
■ Phase 1 制約
  - 基幹システム（販売管理ソフト）との自動連携なし → CSV出力で対応
  - FAX自動受信はFAXサーバー（複合機のメール転送機能）前提
  - 電話注文のSTT対応はPhase 2+
  - 商品マスタは事前登録済みが前提（初回はCSVインポート）

■ 前提
  - 商品マスタにJANコード or 自社商品コードが登録済み
  - 得意先マスタに掛率・締日・支払条件が設定済み
  - FAXはメール転送（TIFF/PDF添付）で受信できる環境
```

---

2. 在庫・倉庫管理

2.1 問題の本質

```
在庫管理の現状:
  - 在庫数がリアルタイムで把握できていない（棚卸しは年1-2回）
  - 「あると思ったらなかった」→ 欠品 → 緊急仕入（高コスト）
  - 「在庫が多すぎて倉庫がパンパン」→ 廃棄ロス（食品卸は特に深刻）
  - 賞味期限管理が手作業（先入れ先出しが守れていない）
  - 在庫問い合わせに営業が電話で答える → 営業の本業圧迫

中小卸の在庫問題:
  過剰在庫コスト = 平均在庫額 × 在庫保管コスト率(15-25%/年)
  1億円の在庫 → 年間1,500-2,500万円の在庫保管コスト
  在庫回転率を6回→8回に改善 → 在庫25%削減 → 年375-625万円のコスト削減
```

2.2 在庫計算式

```
■ ABC分析（パレート分析）

  A品目: 売上高上位20%の商品（全売上の80%を占める）→ 毎日管理
  B品目: 次の30%の商品（全売上の15%）→ 週次管理
  C品目: 残りの50%の商品（全売上の5%）→ 月次管理

  分類基準:
    年間売上金額 = Σ(月間出荷数量 × 単価)
    累積構成比で区分: ～80% = A、80-95% = B、95-100% = C

■ 安全在庫の計算

  安全在庫 = 安全係数(k) × √(発注リードタイム(日)) × 需要の標準偏差(σd)

  安全係数(k)の目安:
    | サービス率 | 安全係数(k) |
    |---|---|
    | 90% | 1.28 |
    | 95% | 1.65 |
    | 97.5% | 1.96 |
    | 99% | 2.33 |
    | 99.5% | 2.58 |

  例: 日販100個、標準偏差30個、リードタイム4日、サービス率95%の場合
    安全在庫 = 1.65 × √4 × 30 = 1.65 × 2 × 30 = 99個

■ 発注点の計算

  発注点 = 平均日販 × 発注リードタイム(日) + 安全在庫

  例: 日販100個、リードタイム4日、安全在庫99個の場合
    発注点 = 100 × 4 + 99 = 499個
    → 在庫が499個を下回ったら発注

■ 経済的発注量（EOQ: Economic Order Quantity）

  EOQ = √(2 × 年間需要量(D) × 1回発注コスト(S) / 年間在庫保管コスト率(H) × 単価(C))

  例: 年間需要36,000個、発注コスト3,000円、保管コスト率20%、単価300円の場合
    EOQ = √(2 × 36,000 × 3,000 / 0.20 × 300) = √(216,000,000 / 60) = √3,600,000 ≒ 1,897個

■ 需要予測

  (a) 移動平均法（直近N期間の平均）
    予測値 = Σ(直近N期の実績) / N
    N=3の場合: (今月 + 先月 + 先々月) / 3

  (b) 指数平滑法（直近データに重みを置く）
    予測値 = α × 今期実績 + (1-α) × 前期予測値
```

```
    α（平滑化係数）= 0.1〜0.3（安定品）/ 0.3〜0.5（変動品）

  (c) 季節指数法（季節変動がある商品）
    季節指数 = 当月平均実績 / 年間月平均実績
    予測値 = ベース予測 × 季節指数
```

2.3 パイプラインステップ

Step	マイクロエージェント	入力	処理	出力
1	run_saas_reader	product_master, inventory_data, io_history	在庫データ読み込み	current_inventory
2	run_cost_calculator	current_inventory + sales_history	ABC分析（商品別年間売上→累積構成比→A/B/C分類）	abc_result
3	run_cost_calculator	sales_history (12-24ヶ月)	需要予測（移動平均/指数平滑/季節指数の自動選択。MAPE評価）	demand_forecast
4	run_cost_calculator	abc_result + demand_forecast	安全在庫・発注点・EOQ計算（ABC別安全係数。発注点割れ→アラート）	reorder_proposals
5	run_rule_matcher	inventory_data (lot/expiry)	賞味期限・ロット管理（30日以内→値引き提案/7日以内→廃棄検討/FIFO出荷チェック）	expiry_alerts
6	run_document_generator	all_inventory_data	在庫レポート生成（在庫金額/在庫回転率/滞留在庫/欠品率）	inventory_report
7	run_output_validator	inventory_report	バリデーション	validated_output

2.4 パイプライン詳細設計（実装）

```
# workers/bpo/wholesale/pipelines/inventory_management_pipeline.py

class InventoryManagementPipeline:
    """
    在庫・倉庫管理パイプライン

    Step 1: inventory_data_reader    在庫データ読み込み
      - マイクロエージェント: micro/data_reader
      - 入力: 商品マスタ、在庫データ、入出庫履歴、発注履歴
      - 出力: 現在の在庫状況一覧

    Step 2: abc_analyzer             ABC分析
      - マイクロエージェント: micro/analyzer
      - 処理:
        (a) 商品別年間売上金額の算出
        (b) 売上金額降順ソート → 累積構成比算出
        (c) ABC分類（A: ～80%、B: 80-95%、C: 95-100%）
        (d) 分類結果を商品マスタに反映
      - 出力: ABC分類結果 + 管理方針提案

    Step 3: demand_forecaster        需要予測
      - マイクロエージェント: micro/demand_forecaster（LLM活用）
      - 処理:
        (a) 過去12-24ヶ月の販売実績データ読み込み
        (b) 商品別に最適な予測手法を自動選択
            - 安定品（変動係数 < 0.3）→ 移動平均法
            - 変動品（変動係数 0.3-0.7）→ 指数平滑法
            - 季節品（季節指数の偏差 > 0.2）→ 季節指数法
        (c) 予測精度の自己評価（MAPE: 平均絶対パーセント誤差）
            MAPE = Σ|実績-予測|/実績 × 100(%) / N
            目標: A品目 MAPE < 20%、B品目 < 30%
      - 出力: 商品別需要予測（翌月～3ヶ月先）

    Step 4: reorder_calculator       安全在庫・発注点計算
      - マイクロエージェント: micro/calculator
      - 処理:
        (a) 商品別の安全在庫を計算
            安全在庫 = k × √(リードタイム) × σd
            k: A品目=1.96(97.5%)、B品目=1.65(95%)、C品目=1.28(90%)
        (b) 発注点を計算
            発注点 = 平均日販 × リードタイム + 安全在庫
        (c) 経済的発注量(EOQ)を計算
        (d) 発注点割れ商品の検出 → 発注アラート生成
      - 出力: 発注提案リスト

    Step 5: expiry_manager           賞味期限・ロット管理
      - マイクロエージェント: micro/expiry_checker
      - 処理:
        (a) 賞味期限切れリスク商品の検出
```

```
            - 30日以内に期限到来 → 値引き販売提案
            - 7日以内 → 廃棄検討アラート
      (b) FIFO出荷チェック（先入れ先出し）
            入荷日/ロット番号の古い順に出荷指示
      (c) ロット別在庫一覧の生成
    - 出力: 期限管理アラート + FIFO出荷指示

  Step 6: report_generator         在庫レポート生成
    - マイクロエージェント: micro/report_generator
    - 処理:
      (a) 在庫金額レポート（商品分類別/倉庫別）
      (b) 在庫回転率レポート
            在庫回転率 = 年間売上原価 / 平均在庫金額
      (c) 滞留在庫レポート（90日以上出荷なし）
      (d) 欠品率レポート
            欠品率 = 欠品件数 / 総受注件数 × 100(%)
    - 出力: 各種在庫レポート

  Step 7: output_validator         バリデーション
  """
```

2.4 データモデル

```
// 商品マスタ
{
  "product_id": "PRD-4901301295095",
  "company_id": "COM-001",
  "product_code": "WH-EX-480R",
  "product_name": "花王 ワイドハイターEX パワー 詰替 480ml",
  "jan_code": "4901301295095",
  "category": "日用品",
  "subcategory": "洗剤・柔軟剤",
  "brand": "花王",
  "unit": "個",
  "case_quantity": 24,
  "inner_quantity": 1,
  "weight_g": 520,
  "dimensions_mm": {"length": 80, "width": 50, "height": 200},
  "cost_price": 220,
  "selling_price": 298,
  "abc_class": "A",
  "shelf_life_days": 1095,
  "temperature_zone": "常温",
  "min_order_quantity": 1,
  "lead_time_days": 3,
  "primary_supplier_id": "SUP-001",
  "status": "販売中"
}

// 在庫データ
{
  "inventory_id": "INV-001",
  "product_id": "PRD-4901301295095",
  "warehouse_id": "WH-001",
  "location": "A-1-03-02",
  "lot_number": "L20260301",
  "expiry_date": "2029-03-01",
  "quantity": 240,
  "allocated_quantity": 48,
  "available_quantity": 192,
  "incoming_quantity": 480,
  "incoming_date": "2026-04-01",
  "cost_price": 220,
  "inventory_value": 52800,
  "last_movement_date": "2026-03-27"
}

// 発注提案
{
  "proposal_id": "PRP-20260328-001",
  "product_id": "PRD-4901301295095",
  "current_stock": 192,
```

```
  "safety_stock": 99,
  "reorder_point": 499,
  "eoq": 1897,
  "proposed_quantity": 1920,
  "proposed_supplier": "SUP-001",
  "estimated_cost": 422400,
  "estimated_delivery": "2026-04-01",
  "reason": "在庫が発注点（499個）を下回りました（現在庫192個）",
  "urgency": "通常"
}
```

---

3. 請求・売掛管理

3.1 問題の本質

```
請求業務の現状:
  - 得意先ごとに締日が違う（末締め/20日締め/15日締め/10日締め）
  - 掛率が得意先ごとに違う（A社は定価の70%、B社は75%...）
  - 月末〜月初の3-5日間は経理が請求書作成に張り付き
  - 入金消込が手作業（振込名義と得意先名が一致しないことが多い）
  - 与信管理が甘く、回収不能が年間数件発生

中小卸の売掛金問題:
  平均売掛金残高: 月商の1.5-2.0ヶ月分
  月商5,000万円 → 売掛金7,500-10,000万円
  貸倒率 0.5% → 年間37.5-50万円の損失
  与信管理強化で貸倒率を0.2%に → 年間22.5-30万円の削減
```

3.2 パイプラインステップ

Step	マイクロエージェント	入力	処理	出力
1	run_saas_reader	sales_details, customer_master	売上データ読み込み（締日別・得意先別集計）	sales_summary
2	run_cost_calculator	sales_summary + customer_master	売価計算（得意先別掛率+数量割引+キャンペーン値引き適用）	priced_details
3	run_document_generator	priced_details	請求書自動生成（インボイス制度対応+電子帳簿保存法対応）	invoice_pdf
4	run_rule_matcher	invoice_pdf + bank_data	売掛金管理（残高更新+入金消込ファジーマッチ+支払期日超過アラート）	receivable_status
5	run_compliance_checker	receivable_status + customer_master	与信管理（与信限度額チェック+支払遅延パターン検出+見直し提案）	credit_report
6	run_output_validator	credit_report	バリデーション	validated_output

3.3 パイプライン詳細設計（実装）

```
# workers/bpo/wholesale/pipelines/billing_pipeline.py

class WholesaleBillingPipeline:
    """
    請求・売掛管理パイプライン

    Step 1: sales_data_reader        売上データ読み込み
      - マイクロエージェント: micro/data_reader
      - 入力: 売上明細、得意先マスタ（締日/掛率/支払条件）
      - 出力: 締日別・得意先別の売上集計

    Step 2: price_calculator         売価計算（掛率適用）
      - マイクロエージェント: micro/price_calculator
      - 処理:
        (a) 得意先別掛率の適用
            売価 = 定価 × 掛率
            ※ 商品別特別単価がある場合はそちらを優先
        (b) 数量割引の適用
            | 数量帯 | 割引率 |
            |---|---|
            | 1-99個 | 0% |
            | 100-499個 | 3% |
            | 500-999個 | 5% |
            | 1,000個以上 | 8% |
        (c) キャンペーン値引きの適用
      - 計算式:
        請求単価 = 定価 × 掛率 × (1 - 数量割引率) × (1 - キャンペーン割引率)
        行合計 = 請求単価 × 数量（端数: 円未満切捨て）
      - 出力: 請求明細（単価確定済み）

    Step 3: invoice_generator        請求書自動生成
      - マイクロエージェント: micro/document_generator
      - 処理:
        (a) 締日別に請求書を生成
            - 前回請求残高 + 当期売上 - 入金 - 値引返品 = 今回請求額
        (b) インボイス制度対応（適格請求書の要件）
            必須記載事項:
            - 適格請求書発行事業者の氏名又は名称及び登録番号（T+13桁）
            - 取引年月日
            - 取引の内容（軽減税率対象品目はその旨）
            - 税率ごとに区分した対価の額及び適用税率
            - 税率ごとに区分した消費税額
            - 書類の交付を受ける事業者の氏名又は名称
            法令参照: 消費税法第57条の4第1項
        (c) 電子帳簿保存法対応
            - 電子発行の場合: タイムスタンプ or 訂正削除履歴
            法令参照: 電子帳簿保存法第7条
      - 出力: 請求書PDF（得意先別）
```

```
Step 4: receivable_manager       売掛金管理
  - マイクロエージェント: micro/receivable_manager
  - 処理:
    (a) 売掛金残高の更新（得意先別・年齢別）
        年齢区分: 当月/1ヶ月超/2ヶ月超/3ヶ月超/6ヶ月超
    (b) 入金消込
        - 振込名義と得意先名のファジーマッチ（Levenshtein距離）
        - 金額完全一致 → 自動消込
        - 金額差異 ≤ 振込手数料（660円） → 手数料差引で自動消込
        - その他 → 手動消込キューに投入
    (c) 支払期日超過アラート
        - 7日超過: 通知
        - 30日超過: 督促
        - 60日超過: 取引停止検討
        - 90日超過: 法的措置検討
  - 出力: 売掛金残高レポート、入金消込結果

Step 5: credit_manager           与信管理
  - マイクロエージェント: micro/credit_checker
  - 処理:
    (a) 取引先別の与信限度額チェック
        与信限度額 = 月間平均取引額 × 与信月数（通常2-3ヶ月）
    (b) 与信超過アラート
        残高が与信限度額の80%超 → 警告
        残高が与信限度額の100%超 → 出荷停止提案
    (c) 支払遅延パターンの検出
        過去12ヶ月の支払遅延回数・日数を分析
        遅延スコア = Σ(遅延日数 × 遅延金額) / 取引総額
    (d) 与信限度額の見直し提案（年1回）
  - 出力: 与信管理レポート、アラート

Step 6: output_validator         バリデーション
"""
```

3.3 データモデル

```
// 得意先マスタ
{
  "customer_id": "CUS-001",
  "company_id": "COM-001",
  "customer_name": "田中商店",
  "customer_code": "T-001",
  "billing_address": "東京都台東区浅草1-2-3",
  "contact_name": "田中太郎",
  "contact_phone": "03-1234-5678",
  "contact_fax": "03-1234-5679",
  "contact_email": "tanaka@example.com",
  "closing_day": 末日,
  "payment_terms": "翌月末払い",
  "payment_method": "銀行振込",
  "discount_rate": 0.30,
  "special_prices": [
    {"product_id": "PRD-001", "special_price": 250, "valid_from": "2026-01-01",
"valid_to": "2026-12-31"}
  ],
  "credit_limit": 5000000,
  "current_receivable": 3200000,
  "invoice_registration_number": "T1234567890123",
  "tax_category": "課税",
  "status": "取引中"
}

// 請求書データ
{
  "invoice_id": "INV-202603-CUS001",
  "customer_id": "CUS-001",
  "customer_name": "田中商店",
  "closing_date": "2026-03-31",
  "payment_due_date": "2026-04-30",
  "previous_balance": 1500000,
  "payments_received": 1500000,
  "current_sales": 1700000,
  "returns_allowances": 50000,
  "current_balance": 1700000,
  "tax_breakdown": {
    "standard_10pct": {"taxable_amount": 800000, "tax_amount": 80000},
    "reduced_8pct": {"taxable_amount": 900000, "tax_amount": 72000}
  },
  "total_with_tax": 1852000,
  "invoice_registration_number": "T9876543210987",
  "items_count": 45,
  "status": "未入金"
}
```

4. 仕入・買掛管理

4.1 問題の本質

```
仕入管理の現状:
  - 発注は電話/FAX/メールでバラバラ
  - 仕入先からの請求書と自社の発注・納品記録の突合が手作業
  - リベート（仕入割戻し）の計算が複雑（年間数百万〜数千万円）
  - 支払予定の管理がExcel（資金繰りの予測ができない）
```

4.2 リベート計算式

```
■ リベート（仕入割戻し）の種類

  (a) 数量リベート
    年間仕入金額に応じた段階的割戻し
    | 年間仕入額 | リベート率 |
    |---|---|
    | 〜1,000万円 | 0% |
    | 1,000〜3,000万円 | 1.0% |
    | 3,000〜5,000万円 | 1.5% |
    | 5,000万円超 | 2.0% |

  (b) 達成リベート
    目標仕入額を達成した場合の一括割戻し
    目標達成率 100%以上 → 仕入額の0.5%

  (c) 早期支払リベート
    支払期日より早く支払った場合の割引
    10日以上早期支払 → 請求額の1.0%

  (d) 新商品導入リベート
    新商品の初回導入時の割戻し
    初回仕入額の5-10%

■ リベート計算タイミング
  - 数量リベート: 年度末（3月末）に確定 → 翌月精算
  - 達成リベート: 半期末（9月/3月）に確定
  - 早期支払リベート: 各支払時に即時適用
```

4.3 パイプラインステップ

Step	マイクロエージェント	入力	処理	出力
1	run_saas_reader	purchase_data, supplier_master, purchase_history	仕入データ読み込み	purchase_status
2	run_document_generator	reorder_proposals + supplier_master	発注書自動生成（仕入先別集約+最低発注金額チェック+PDF/FAX出力）	purchase_orders
3	run_diff_detector	delivery_data + purchase_orders	検品・検収処理（品目/数量/単価の突合。差異±3%以内=OK）	receiving_result
4	run_document_ocr + run_rule_matcher	supplier_invoices + receiving_data	請求書照合（OCR読取→自社仕入データ突合。差異パターン検出）	invoice_match
5	run_cost_calculator	supplier_purchase_totals + rebate_conditions	リベート計算（数量/達成/早期支払/新商品。年間見込み額予測）	rebate_result
6	run_cost_calculator	invoice_match + rebate_result	買掛金管理・支払予定（残高更新+支払予定表+資金繰り予測+早期支払判断）	payment_schedule
7	run_output_validator	payment_schedule	バリデーション	validated_output

4.4 パイプライン詳細設計（実装）

```
# workers/bpo/wholesale/pipelines/purchasing_pipeline.py

class PurchasingPipeline:
    """
    仕入・買掛管理パイプライン

    Step 1: purchase_data_reader     仕入データ読み込み
      - マイクロエージェント: micro/data_reader
      - 入力: 発注データ、仕入先マスタ、仕入実績
      - 出力: 仕入状況一覧

    Step 2: order_generator          発注書自動生成
      - マイクロエージェント: micro/document_generator
      - 処理:
        (a) 在庫パイプラインの発注提案から発注書を生成
        (b) 仕入先別に集約（同一仕入先の複数商品を1発注にまとめる）
        (c) 最低発注金額チェック（仕入先の最低発注条件）
        (d) 発注書PDF/FAX/メール出力
      - 出力: 発注書

    Step 3: receiving_inspector      検品・検収処理
      - マイクロエージェント: micro/inspector
      - 処理:
        (a) 納品データと発注データの突合
            - 品目一致チェック
            - 数量一致チェック（±3%以内は許容 = 検収OK）
            - 単価一致チェック
        (b) 差異がある場合 → 差異レポート + 仕入先への連絡文書生成
        (c) 検収OK → 在庫に反映（入庫処理）
      - 出力: 検収結果

    Step 4: invoice_matcher          請求書照合
      - マイクロエージェント: micro/invoice_matcher（OCR + LLM活用）
      - 処理:
        (a) 仕入先請求書のOCR読み取り
        (b) 自社の仕入データ（検収済み）との突合
            - 金額一致 → 自動承認
            - 差異あり → 差異明細レポート + 要確認キュー
        (c) 差異パターン:
            - 単価違い（契約単価と異なる）
            - 数量違い（検収数量と異なる）
            - 未納品の請求（納品前に請求が来ている）
            - 二重請求（同じ内容が2回請求されている）
      - 出力: 照合結果 + 差異レポート

    Step 5: rebate_calculator        リベート計算
      - マイクロエージェント: micro/rebate_calculator
      - 処理:
        (a) 仕入先別の年間（半期）仕入累計額を集計
```

```
        (b) リベート条件テーブルに基づく割戻し額の計算
        (c) リベート見込み額の表示（年度途中での予測）
            現在の仕入ペース × 残月数 → 年間予測 → リベート率判定
      - 出力: リベート計算結果 + 見込みレポート

    Step 6: payment_scheduler        買掛金管理・支払予定
      - マイクロエージェント: micro/payment_manager
      - 処理:
        (a) 買掛金残高の更新（仕入先別）
        (b) 支払予定表の生成（日次/週次/月次）
        (c) 資金繰り予測
            予測残高 = 現在残高 + 入金予定 − 支払予定
        (d) 早期支払リベートの適用判断
            早期支払割引額 > 資金コスト(借入金利) → 早期支払推奨
      - 出力: 支払予定表、資金繰り予測

    Step 7: output_validator         バリデーション
    """
```

---

5. 物流・配送管理

5.1 問題の本質

```
配送管理の現状:
  - 出荷指示書を手作業で作成（受注データからコピー）
  - 送り状（伝票番号）の入力が手作業
  - 配送状況の問い合わせに「確認します」→ 折り返し
  - 自社便と宅配便の使い分けが曖昧（コスト最適化されていない）
```

5.2 配送コスト最適化計算

```
■ 自社便 vs 宅配便の判定

  自社便コスト = 固定費（車両+人件費）/ 日あたり配送件数 + 変動費（燃料）
    例: 月60万円(固定) / 22日 / 15件 = 1,818円/件 + 燃料500円 ≒ 2,300円/件

  宅配便コスト = サイズ別運賃（法人契約単価）
    | サイズ | 重量 | 運賃（関東圏内） |
    |---|---|---|
    | 60 | ～2kg | 650円 |
    | 80 | ～5kg | 850円 |
    | 100 | ～10kg | 1,050円 |
    | 120 | ～15kg | 1,300円 |
    | 140 | ～20kg | 1,600円 |
    | 160 | ～25kg | 1,900円 |

  判定ルール:
    1件あたりの荷物が小型（120サイズ以下）→ 宅配便が有利
    大口配送（パレット単位）→ 自社便 or 路線便が有利
    緊急配送 → 自社便
    遠距離 → 宅配便 or 路線便

■ 送り状発行API

  ヤマト運輸: B2クラウドAPI
  佐川急便: e飛伝API
  日本郵便: ゆうパックプリントAPI
  → 受注データから自動で送り状データを生成・送信
```

5.3 パイプラインステップ

Step	マイクロエージェント	入力	処理	出力
1	run_saas_reader	order_data + inventory_allocation	受注データ読み込み（出荷対象リスト生成）	shipping_targets
2	run_cost_calculator	shipping_targets + delivery_addresses	配送方法最適化（自社便vs宅配便判定+最安キャリア選定）	optimized_shipping
3	run_document_generator	optimized_shipping + inventory_locations	ピッキングリスト生成（ロケーション順ソート+FIFO指示+欠品アラート）	picking_list
4	run_document_generator	optimized_shipping	出荷書類自動生成（出荷指示書+納品書+送り状CSV+物流ラベル）	shipping_documents
5	run_saas_reader	tracking_numbers	配送状況追跡（各社追跡API→配達完了/遅延/不在持戻りの検知）	tracking_status
6	run_output_validator	tracking_status	バリデーション	validated_output

5.4 パイプライン詳細設計（実装）

```
# workers/bpo/wholesale/pipelines/shipping_pipeline.py

class ShippingPipeline:
    """
    物流・配送管理パイプライン

    Step 1: order_data_reader        受注データ読み込み
      - マイクロエージェント: micro/data_reader
      - 入力: 出荷対象の受注データ、在庫引当結果
      - 出力: 出荷指示対象リスト

    Step 2: shipping_optimizer       配送方法最適化
      - マイクロエージェント: micro/shipping_optimizer
      - 処理:
        (a) 配送先別に最適な配送方法を判定
            - 近距離（同一都道府県）+ 大口 → 自社便
            - 近距離 + 小口 → 宅配便
            - 遠距離 → 宅配便 or 路線便
            - 物流パイプライン連携（物流BPOを導入している場合は配車連携）
        (b) 自社便の場合: ルート最適化（物流パイプライン#1と連携可能）
        (c) 宅配便の場合: 最安キャリアの選定
      - 出力: 配送方法指定済み出荷指示

    Step 3: picking_list_generator   ピッキングリスト生成
      - マイクロエージェント: micro/document_generator
      - 処理:
        (a) 出荷指示からピッキングリストを生成
        (b) ロケーション順にソート（倉庫内の動線最適化）
        (c) FIFO指示（賞味期限の古いロットから出庫）
        (d) 欠品商品のアラート
      - 出力: ピッキングリスト

    Step 4: document_generator       出荷書類自動生成
      - マイクロエージェント: micro/document_generator
      - 処理:
        (a) 出荷指示書の生成
        (b) 納品書の生成（得意先別フォーマット対応）
        (c) 送り状データの生成
            - ヤマト: B2クラウドCSVフォーマット
            - 佐川: e飛伝CSVフォーマット
            - 日本郵便: ゆうプリRCSVフォーマット
        (d) 物流ラベルの生成（バーコード/QR付き）
      - 出力: 出荷書類一式 + 送り状CSV

    Step 5: tracking_manager         配送状況追跡
      - マイクロエージェント: micro/tracking_manager
      - 処理:
        (a) 送り状番号から配送状況をAPI取得
            - ヤマト: 荷物追跡API
```

```
            - 佐川: 荷物追跡API
            - 日本郵便: 配達状況確認API
        (b) 配達完了 → 納品ステータス更新
        (c) 配達遅延 → アラート + 得意先への通知提案
        (d) 不在持戻り → 再配達手配提案
      - 出力: 配送状況レポート

    Step 6: output_validator         バリデーション
    """
```

---

6. 営業支援（卸売特化）

6.1 問題の本質

```
卸売営業の現状:
  - 「御用聞き営業」が主流。得意先を巡回して注文を聞くだけ
  - 商品知識は営業個人の経験頼み
  - どの得意先が儲かっているか（粗利貢献）が不明
  - 季節商品の提案タイミングを逃す
  - 新商品の案内が全得意先一律（ニーズに合っていない）
```

6.2 分析計算式

```
■ 取引先別売上分析

  (a) RFM分析（顧客ランク分類）

    ■ スコアリング定義
    R_score（Recency: 最終購入日からの経過日数）:
      5段階: 30日以内=5, 31-60日=4, 61-90日=3, 91-180日=2, 181日以上=1

    F_score（Frequency: 月間購入回数）:
      5段階: 月4回以上=5, 月3回=4, 月2回=3, 月1回=2, 月1回未満=1

    M_score（Monetary: 月間購入金額）:
      5段階: 100万以上=5, 50-100万=4, 20-50万=3, 5-20万=2, 5万未満=1

    ■ 総合スコア計算式
    RFM_total = R_score × w_R + F_score × w_F + M_score × w_M
    デフォルト重み: w_R=0.3, w_F=0.3, w_M=0.4
    ※ 重みは業態・戦略に応じて調整可能

    ■ ランク判定（RFM_total → 顧客ランク）
    RFM_total ≥ 4.5 → Aランク（最優良顧客）→ 重点フォロー
    RFM_total ≥ 3.5 → Bランク（優良顧客）→ 維持施策
    RFM_total ≥ 2.5 → Cランク（一般顧客）→ 育成施策
    RFM_total < 2.5 → Dランク（休眠/低活性顧客）→ 復活施策 or 撤退判断

    ■ 特殊パターン検知
    R=5, F=5, M=1-2 → 成長顧客（頻繁だが単価低い。拡大施策）
    R=1-2, F=4-5, M=4-5 → 離反リスク（かつて優良。緊急フォロー）
    R=5, F=1-2, M=5 → 大口スポット（大型案件の機会。関係深耕）

    ■ 計算例
    田中商店: R=5(最終購入2日前), F=4(月3回), M=4(月75万円)
    RFM_total = 5×0.3 + 4×0.3 + 4×0.4 = 1.5 + 1.2 + 1.6 = 4.3 → Bランク

    優良顧客: R1+F1+M1 → 重点フォロー
    休眠顧客: R4 → 復活施策
    成長顧客: R1+F1+M3 → 拡大施策

  (b) 粗利分析
    粗利額 = 売上額 - 仕入原価（仕入単価 × 数量）
    粗利率 = 粗利額 / 売上額 × 100(%)
    ※ 得意先別・商品別のマトリクスで「高売上・低粗利」を特定

■ 商品別粗利分析（クロスABC分析）

  売上ABC × 粗利ABC のマトリクス:
    | | 粗利A | 粗利B | 粗利C |
    |---|---|---|---|
    | 売上A | ★最重要（守る） | 粗利改善 | 値上げ交渉 |
```

```
    | 売上B | 拡販推進 | 通常管理 | 見直し |
    | 売上C | ニッチ強化 | 自然減容認 | 廃番候補 |

■ 購買パターン分析（アソシエーション分析）

  支持度(Support) = 商品Aと商品Bを同時に購入した取引数 / 全取引数
  信頼度(Confidence) = 商品Aを購入した場合に商品Bも購入する確率
  リフト値(Lift) = Confidence / 商品B単独の購入確率

  リフト値 > 1.5 → 有意な関連あり → クロスセル推奨
  例: 「食器用洗剤を購入した得意先の65%がスポンジも購入」→ セット提案

■ 季節需要予測

  季節指数 = 当月の過去3年平均売上 / 年間月平均売上
  例: 殺虫剤
    1月: 0.3、2月: 0.4、3月: 0.6、4月: 1.0、5月: 1.5、6月: 2.0
    7月: 2.5、8月: 2.3、9月: 1.5、10月: 0.8、11月: 0.4、12月: 0.3
  → 6-7月にピーク。4月から提案開始
```

6.3 パイプラインステップ

Step	マイクロエージェント	入力	処理	出力
1	run_saas_reader	sales_records, customer_master, product_master, purchase_data	売上・取引データ読み込み	analysis_dataset
2	run_cost_calculator	analysis_dataset	取引先分析（RFMスコア計算→ランク分類+売上推移+粗利分析+離反リスク検知）	customer_analysis
3	run_cost_calculator	analysis_dataset	商品分析（売上ABC×粗利ABCクロス分析+トレンド分析+廃番候補抽出）	product_analysis
4	run_structured_extractor	customer_analysis + product_analysis	推奨商品エンジン（アソシエーション分析→クロスセル+季節需要先行提案+新商品マッチング）	recommendations
5	run_document_generator	recommendations + kpi_data	営業レポート生成（担当者別KPI+得意先別アクション提案+月次会議用レポート）	sales_report
6	run_output_validator	sales_report	バリデーション	validated_output

6.4 パイプライン詳細設計（実装）

```
# workers/bpo/wholesale/pipelines/sales_support_pipeline.py

class WholesaleSalesSupportPipeline:
    """
    営業支援パイプライン（卸売特化）

    Step 1: sales_data_reader        売上・取引データ読み込み
      - マイクロエージェント: micro/data_reader
      - 入力: 売上実績、得意先マスタ、商品マスタ、仕入データ
      - 出力: 分析用データセット

    Step 2: customer_analyzer        取引先分析
      - マイクロエージェント: micro/analyzer
      - 処理:
        (a) RFM分析 → 顧客ランク分類
        (b) 得意先別売上推移（前年比、前月比）
        (c) 得意先別粗利分析
        (d) 取引頻度の変化検知（減少トレンド → 離反リスクアラート）
      - 出力: 取引先分析レポート

    Step 3: product_analyzer         商品分析
      - マイクロエージェント: micro/analyzer
      - 処理:
        (a) 商品別売上ABC分析
        (b) 商品別粗利分析 + クロスABC
        (c) 商品カテゴリ別のトレンド分析
        (d) 廃番候補の抽出（売上C × 粗利C × 6ヶ月出荷なし）
      - 出力: 商品分析レポート

    Step 4: recommendation_engine    推奨商品エンジン
      - マイクロエージェント: micro/recommender（LLM活用）
      - 処理:
        (a) アソシエーション分析 → クロスセル推奨
            「この得意先は商品Aを購入していますが、商品Bはまだです。
             同業態の他社は65%が商品Bも購入しています」
        (b) 季節需要予測に基づく先行提案
            「来月から殺虫剤の需要が上がります。今のうちに案内しましょう」
        (c) 新商品マッチング
            得意先の購買カテゴリ → 新商品のカテゴリが一致 → 提案
        (d) 購買頻度・数量の異常検知
            いつもの注文が来ない → 「田中商店の月曜注文が未着です」
      - 出力: 推奨アクションリスト

    Step 5: report_generator         営業レポート生成
      - マイクロエージェント: micro/report_generator
      - 処理:
        (a) 営業担当者別のKPIダッシュボード
            - 売上達成率、粗利率、新規開拓数、訪問件数
        (b) 得意先別の営業アクション提案
```

```
        (c) 月次営業会議用レポート
    - 出力: 各種営業レポート

  Step 6: output_validator         バリデーション
  """
```

6.4 データモデル

```
// 営業推奨アクション
{
  "recommendation_id": "REC-20260328-001",
  "customer_id": "CUS-001",
  "customer_name": "田中商店",
  "action_type": "cross_sell",
  "priority": "high",
  "title": "スポンジのクロスセル提案",
  "description": "田中商店は食器用洗剤を月3ケース購入していますが、スポンジは未購入です。同業態の小売店の65%がスポンジも同時購入しています。",
  "recommended_products": [
    {
      "product_id": "PRD-SP-001",
      "product_name": "キクロンA 5個入",
      "estimated_monthly_quantity": "2ケース",
      "estimated_monthly_revenue": 8400,
      "estimated_margin_rate": 0.22,
      "confidence": 0.78
    }
  ],
  "basis": {
    "analysis_type": "association_rule",
    "support": 0.12,
    "confidence": 0.65,
    "lift": 2.1,
    "sample_size": 450
  },
  "suggested_timing": "次回訪問時（4/2予定）",
  "status": "未対応"
}

// 季節需要予測
{
  "forecast_id": "FCT-202604",
  "product_id": "PRD-INS-001",
  "product_name": "アース ブラックキャップ 12個入",
  "category": "殺虫剤",
  "month": "2026-04",
  "forecast_quantity": 150,
  "forecast_method": "seasonal_index",
  "seasonal_index": 1.0,
  "peak_month": "2026-07",
  "peak_seasonal_index": 2.5,
  "recommendation": "4月から店頭展開の提案を開始。6月までに在庫を確保（ピーク月需要: 375個）",
  "confidence": 0.82
}
```

既存パイプラインとの共通性

卸売業モジュール	再利用元	共通度	追加開発
受発注AI	construction/estimation（OCR→抽出→構造化）	80%	商品マスタ照合、FAX特化OCR
在庫・倉庫管理	logistics/warehouse（入出庫管理）	60%	需要予測、ABC分析、賞味期限管理
請求・売掛管理	construction/billing（請求書照合）	70%	掛率計算、入金消込、与信管理
仕入・買掛管理	construction/subcontractor（下請管理）	50%	検品処理、リベート計算
物流・配送管理	logistics/dispatch（配車計画）	40%	送り状API連携、ピッキングリスト
営業支援	brain/knowledge/qa（Q&A）	30%	RFM分析、クロスセル推奨エンジン

結論: 受発注AI・請求照合は既存パイプラインの70-80%を再利用可能。卸売固有のゲノムJSON定義 + 商品マスタ照合ロジック + 需要予測が主な追加開発。

DB設計（追加テーブル）

```
-- 商品マスタ（卸売業）
CREATE TABLE wholesale_products (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  product_code TEXT NOT NULL,
  product_name TEXT NOT NULL,
  jan_code TEXT,
  category TEXT,
  subcategory TEXT,
  brand TEXT,
  unit TEXT NOT NULL DEFAULT '個',          -- 個/本/箱/袋/缶/L/kg
  case_quantity INTEGER DEFAULT 1,          -- 1ケースあたりの入数
  inner_quantity INTEGER DEFAULT 1,         -- 1インナーあたりの入数
  weight_g INTEGER,
  dimensions_mm JSONB,                      -- {"length": 80, "width": 50, "height": 200}
  cost_price DECIMAL(10,2),                 -- 仕入単価
  selling_price DECIMAL(10,2),              -- 定価（標準売価）
  abc_class TEXT DEFAULT 'C',               -- A/B/C
  shelf_life_days INTEGER,                  -- 賞味期限（日数）
  temperature_zone TEXT DEFAULT '常温',     -- 常温/冷蔵/冷凍
  min_order_quantity INTEGER DEFAULT 1,
  lead_time_days INTEGER DEFAULT 3,
  primary_supplier_id UUID,
  reorder_point INTEGER,                    -- 発注点
  safety_stock INTEGER,                     -- 安全在庫
  eoq INTEGER,                              -- 経済的発注量
  status TEXT DEFAULT '販売中',             -- 販売中/廃番予定/廃番
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- 得意先マスタ（卸売業）
CREATE TABLE wholesale_customers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  customer_code TEXT NOT NULL,
  customer_name TEXT NOT NULL,
  customer_name_kana TEXT,
  business_type TEXT,                       -- 小売店/飲食店/事業所/他卸
  billing_address TEXT,
  shipping_address TEXT,
  contact_name TEXT,
  contact_phone TEXT,
  contact_fax TEXT,
  contact_email TEXT,
  closing_day INTEGER NOT NULL DEFAULT 31,  -- 締日（10/15/20/25/末=31）
  payment_terms TEXT DEFAULT '翌月末',      -- 翌月末/翌々月末/都度
  payment_method TEXT DEFAULT '銀行振込',   -- 銀行振込/手形/口座振替/現金
```

```
  discount_rate DECIMAL(4,3) DEFAULT 0.000, -- 掛率からの割引（例: 0.30 = 定価の70%）
  credit_limit INTEGER DEFAULT 0,           -- 与信限度額（円）
  invoice_registration_number TEXT,          -- インボイス登録番号 T+13桁
  rfm_rank TEXT DEFAULT 'C',                -- A/B/C/D（RFM分析結果）
  sales_rep TEXT,                            -- 担当営業
  status TEXT DEFAULT '取引中',
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- 仕入先マスタ
CREATE TABLE wholesale_suppliers (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  supplier_code TEXT NOT NULL,
  supplier_name TEXT NOT NULL,
  contact_name TEXT,
  contact_phone TEXT,
  contact_fax TEXT,
  contact_email TEXT,
  payment_terms TEXT DEFAULT '月末締め翌月末払い',
  lead_time_days INTEGER DEFAULT 3,
  min_order_amount INTEGER DEFAULT 0,       -- 最低発注金額
  rebate_conditions JSONB DEFAULT '[]',     -- [{type, threshold, rate}]
  status TEXT DEFAULT '取引中',
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- 受注データ
CREATE TABLE wholesale_orders (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  order_number TEXT NOT NULL,
  customer_id UUID NOT NULL,
  order_date DATE NOT NULL,
  desired_delivery_date DATE,
  source_type TEXT DEFAULT 'manual',        -- fax/email/line/phone/manual/edi
  source_document_url TEXT,
  ocr_confidence DECIMAL(3,2),
  status TEXT DEFAULT 'pending',            --
pending/confirmed/picked/shipped/delivered/cancelled
  subtotal INTEGER DEFAULT 0,
  tax_amount INTEGER DEFAULT 0,
  total_amount INTEGER DEFAULT 0,
  notes TEXT,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- 受注明細
CREATE TABLE wholesale_order_items (
```

```
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  order_id UUID NOT NULL REFERENCES wholesale_orders(id) ON DELETE CASCADE,
  company_id UUID NOT NULL REFERENCES companies(id),
  line_no INTEGER NOT NULL,
  product_id UUID,
  product_code TEXT,
  product_name TEXT NOT NULL,
  quantity INTEGER NOT NULL,
  unit TEXT NOT NULL,
  unit_price DECIMAL(10,2),
  amount INTEGER GENERATED ALWAYS AS (ROUND(quantity * COALESCE(unit_price, 0))) STORED,
  match_confidence DECIMAL(3,2),            -- 商品マッチの確信度
  match_type TEXT,                           -- exact/fuzzy/embedding/history/manual
  lot_number TEXT,
  expiry_date DATE,
  status TEXT DEFAULT 'pending',
  created_at TIMESTAMPTZ DEFAULT now()
);

-- 在庫テーブル
CREATE TABLE wholesale_inventory (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  product_id UUID NOT NULL,
  warehouse_id TEXT DEFAULT 'main',
  location TEXT,                             -- ロケーション（棟-階-列-段）
  lot_number TEXT,
  expiry_date DATE,
  quantity INTEGER NOT NULL DEFAULT 0,
  allocated_quantity INTEGER NOT NULL DEFAULT 0, -- 引当済み
  available_quantity INTEGER GENERATED ALWAYS AS (quantity - allocated_quantity) STORED,
  cost_price DECIMAL(10,2),
  last_movement_date TIMESTAMPTZ,
  created_at TIMESTAMPTZ DEFAULT now(),
  updated_at TIMESTAMPTZ DEFAULT now()
);

-- 商品名表記揺れ辞書（自動学習）
CREATE TABLE product_name_aliases (
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  company_id UUID NOT NULL REFERENCES companies(id),
  alias_text TEXT NOT NULL,                  -- 表記揺れテキスト（「WH詰替」等）
  product_id UUID NOT NULL,                  -- 正規化先の商品ID
  customer_id UUID,                          -- 得意先固有の場合（NULLは全社共通）
  match_count INTEGER DEFAULT 1,             -- マッチ回数（学習データ）
  created_at TIMESTAMPTZ DEFAULT now(),
  UNIQUE(company_id, alias_text, customer_id)
);

-- RLS
ALTER TABLE wholesale_products ENABLE ROW LEVEL SECURITY;
ALTER TABLE wholesale_customers ENABLE ROW LEVEL SECURITY;
```

```
ALTER TABLE wholesale_suppliers ENABLE ROW LEVEL SECURITY;
ALTER TABLE wholesale_orders ENABLE ROW LEVEL SECURITY;
ALTER TABLE wholesale_order_items ENABLE ROW LEVEL SECURITY;
ALTER TABLE wholesale_inventory ENABLE ROW LEVEL SECURITY;
ALTER TABLE product_name_aliases ENABLE ROW LEVEL SECURITY;

-- ========================================
-- 卸売業BPO テナント分離RLSポリシー
-- 全テーブル共通: company_id ベースのRLS
-- ========================================

CREATE POLICY wholesale_products_tenant ON wholesale_products
    USING (company_id = current_setting('app.company_id', true)::UUID);
CREATE POLICY wholesale_customers_tenant ON wholesale_customers
    USING (company_id = current_setting('app.company_id', true)::UUID);
CREATE POLICY wholesale_suppliers_tenant ON wholesale_suppliers
    USING (company_id = current_setting('app.company_id', true)::UUID);
CREATE POLICY wholesale_orders_tenant ON wholesale_orders
    USING (company_id = current_setting('app.company_id', true)::UUID);
CREATE POLICY wholesale_order_items_tenant ON wholesale_order_items
    USING (company_id = current_setting('app.company_id', true)::UUID);
CREATE POLICY wholesale_inventory_tenant ON wholesale_inventory
    USING (company_id = current_setting('app.company_id', true)::UUID);
CREATE POLICY product_name_aliases_tenant ON product_name_aliases
    USING (company_id = current_setting('app.company_id', true)::UUID);
```

法令参照

■ 下請代金支払遅延等防止法（下請法）
- 第2条の2: 下請代金の支払期日（受領日から起算して60日以内）
- 第3条: 書面の交付義務（発注時に注文書を書面交付する義務）
- 第4条: 親事業者の禁止行為
  - 第1項第1号: 受領拒否の禁止
  - 第1項第2号: 下請代金の支払遅延の禁止
  - 第1項第3号: 下請代金の減額の禁止
  - 第1項第5号: 買いたたきの禁止
  - 第2項第1号: 不当な返品の禁止
  - 第2項第3号: 不当なやり直し等の禁止

■ 独占禁止法（私的独占の禁止及び公正取引の確保に関する法律）
- 第2条第9項第5号: 優越的地位の濫用
  （仕入先・得意先との取引で不当に有利な条件を押し付ける行為の禁止）
- 公正取引委員会「優越的地位の濫用に関する独占禁止法上の考え方」
  （リベート交渉・仕入条件変更時の参照基準）

■ 消費税法
- 第57条の4: 適格請求書発行事業者の義務（インボイス制度）
  - 第1項: 適格請求書の記載事項（登録番号、税率区分、消費税額等）
  - 適格返還請求書（返品・値引き時の記載義務）

※ 2023年10月施行。仕入税額控除の要件として適格請求書の保存が必須

■ 電子帳簿保存法
- 第7条: 電子取引データの保存義務
  - 電子発行した請求書・領収書の電子保存（紙保存不可。2024年1月完全義務化）
  - タイムスタンプ or 訂正削除履歴の保存要件
- 第4条: 帳簿の電磁的記録による保存

※ 請求書PDF・受注確認書PDFの発行時に電子帳簿保存法対応が必要

■ 食品衛生法（食品卸の場合）
- 第52条: 営業の許可（食品の販売業は許可 or 届出が必要）
- 第55条: 営業の届出
- 第58条: 食品等の回収（リコール時の届出義務。2021年6月施行）
- HACCP に沿った衛生管理の義務化（2021年6月完全施行）

※ 食品卸は温度管理・賞味期限管理・ロット追跡が法的義務

■ 景品表示法
- 第5条: 不当な表示の禁止（商品カタログ・販促資料のチェック）

■ 製造物責任法（PL法）
- 卸売業者は直接の責任主体ではないが、輸入品の場合は輸入者として責任を負う

※ 仕入先管理・品質管理の観点でPL保険の確認が必要

実装ロードマップ

```
Phase A（Week 1–2）: ★最優先
  → #1 受発注AI（OCR + LLM構造化抽出 + 商品マスタ照合）
  → #2 在庫・倉庫管理（ABC分析 + 安全在庫 + 発注点 + 賞味期限管理）

Phase B（Week 3）:
  → #3 請求・売掛管理（締日別請求書 + 入金消込 + 与信管理）
  → #4 仕入・買掛管理（発注書自動生成 + 請求書照合 + リベート計算）

Phase C（Week 4）:
  → #5 物流・配送管理（出荷指示 + 送り状発行 + 配送追跡）
  → #6 営業支援（RFM分析 + クロスセル推奨 + 季節需要予測）
```